Programowanie Współbieżne

C#

Wait Handles

- Win32 Api dostarcza trzy klasy
  - `EventWaitHandle`
  - `Mutex`
  - `Semaphore`
- Wszystkie trzy bazują na abstrakcyjnej klasie `WaitHandle`
- `EventWaitHandle` ma dwie podklasy
  - `AutoResetEvent`
  - `ManualResetEvent`
- Różnica jest właściwie tylko w sposobie wołania konstrukora
- `WaitHandles` pozwalają na nazwanie klas i używanie pomiędzy odrębnymi procesami

Wait Handles

`AutoResetEvent`

- Można porównać do bramki która przepuszcza tylko jeden proces za naciśnięciem jednego guzika.
- Gdy bramka jest otwarta proces lub wątek który wywoła metodę `WaitOne()` przechodzi przez bramkę jednocześnie ją zamykając
- Gdy bramka jest zamknięta proces ustawia się w kolejce.
- Każdy inny nie zablokowany proces może odblokować bramkę za pomocą wywołania metody `Set()`
- Jedno wywołanie `Set()` wpuści tylko jeden proces.
- Gdy nie będzie procesów w kolejce, `Set()` otworzy bramkę
- Gdy bramka jest już otwarta, następne `Set()` są ignorowane.

Wait Handles

AutoResetEvent

```
EventWaitHandle waitingGate = new EventWaitHandle (false,
                                    EventResetMode.Auto);
EventWaitHandle waitingGate = new AutoResetEvent (false);
```

- Powyższe dwa wywołania są równoważne.
- Pierwszy parametr określa czy bramka ma być podczas utworzenia otwarta.

Przykład (WaitHandles)

Wait Handles

`EventWaitHandle` - między procesami

```
EventWaitHandle waitingGate = new EventWaitHandle (false,
            EventResetMode.Auto,"Name of our waitingGate");
```

- Trzecim parametrem może być nazwa widziana przez wszystkie inne procesy w systemie.
- Gdy podczas tworzenia okaże sie, że obiekt o podanej nazwie istnieje dostaniemy tylko referencję a czwarty parametr będzie false;

```
EventWaitHandle (false, EventResetMode.Auto,"Name of our
waitingGate", out isNew);
```

Ready Go

Załóżmy, że mamy taki scenariusz

- Główny proces ma co chwilę nowe zadania do wykonania
- Zadania te mają być wykonane przez wątek
- Za każdym razem uruchamiany jest nowy wątek
- Przekazywane jest zadanie
- Po wykonaniu pracy wątek jest kończony

By zmniejszyć obciążenie wynikające z tworzenia wątków (czy nawet innych procesów) możemy postępować według poniższego algorytmu:

- Główny proces tworzy wątek
- Wątek czeka na zadanie
- Wykonuje zadanie
- Przechodzi w stan oczekiwania na kolejne zadanie

Ready Go

- Najprostsza wersja producenta i konsumenta

```
static EventWaitHandle ready = new AutoResetEvent(false);
static EventWaitHandle go = new AutoResetEvent(false);
static volatile string job;
static void Main(string[] args)
{
    new Thread(Consumer).Start();
    for (int i = 1; i <= 5; i++) //order job 5 times
        {
        ready.WaitOne(); // Waiting for consumer ready
        job = "a".PadRight(i, 'a'); // prepare job
        go.Set(); // signal job are ready to read
        }
    ready.WaitOne(); job = null; go.Set(); // it is signal to end
    Console.ReadKey();
    }
static void Consumer()
    {
    while (true)
        {
        ready.Set(); // Inform producer we are ready
        go.WaitOne(); // and waiting for a job
        if (job == null) return; // when we get null we ends
            Console.WriteLine(job);
        }
    }
```

example ReadyGo

Kolejka producent - konsument

- Wykorzystanie procesu drugoplanowego
- Producent kolejkuje elementy
- Konsument dekolejkuje elementy
- Rozwiązanie podobne do poprzedniego tylko nie blokujące

```
static void Main(string[] args)
        {
            Queue<string> queue = new Queue<string>();
            Thread producerThread = Producer.StartProduction(queue);
            Thread consumerThread = Consumer.StartConsumption(queue);
            producerThread.Join();
            consumerThread.Join();
        }
```

Kolejka producent - konsument

```
public static Thread StartProduction(Queue<string> queue)
    {
    if (producerThread == null)
        {
        producerThread = new Thread((q) =>
            {
            Random r = new Random();
            for (int i = 0; i < 100; i++)
                {
                Thread.Sleep(r.Next(0, 30));
                lock (q)
                    {
                    (q as Queue<string>).Enqueue("This is a product no " + i);
                    Console.WriteLine("P: I'v put to the queue");
                    }
                 wh.Set();
                 }
             lock (q)
                {
                (q as Queue<string>).Enqueue(null);
                }
         });
         producerThread.Start(queue);
        }
    else
        {
        Console.WriteLine("There is already one thread");
        }
    return producerThread;
    }
```

Kolejka producent - konsument

```
public static Thread StartConsumption(Queue<string> queue)
     {
         if (consumerThread == null)
         {
             consumerThread = new Thread((q) =>
             {
                 Random r = new Random();
                 while (true)
                 {
                     string mesg = null;
                     lock (q)
                         if ((q as Queue<string>).Count > 0)
                         {
                             mesg = (q as Queue<string>).Dequeue();
                             if (mesg == null) return;
                         }
                     if (mesg != null)
                     {
                         Console.WriteLine("C: I'v consumed: " + mesg);
                         Thread.Sleep(r.Next(0, 20));
                     }
                     else
                     {
                         //sometimes it happens twice
                         Console.WriteLine("C: So I'm waiting...");
                         wh.WaitOne();
                     }
                 }
             });
             consumerThread.Start(queue);
         }
         else
         {
             Console.WriteLine("There is already one thread");
         }
         return consumerThread;
 }
```

Wait Handles

`ManualResetEvent`

```
EventWaitHandle waitingGate = new EventWaitHandle (false,
  EventResetMode.Manual);

EventWaitHandle waitingGate = new ManualResetEvent (false);
```

- Powyższe dwa wywołania są równoważne.
- Pierwszy parametr określa czy bramka ma być podczas utworzenia otwarta.
- Metoda `Set` wpuszcza wszystkich czekających lub wołających `WaitOne` dopóki nie zamknie się przez `Reset`.

.

Wait Handles

`Mutex`

- Działa tak samo jak `lock` z tym że może być używany pomiędzy procesami i jest około 100 razy wolniejszy (przy założeniu że nie blokujemy)
- Tak samo jak lock zapewnia wyłączny dostęp do bloku programu pomiędzy wywołaniem WaitOne a ReleaseMutex i musi być wywołany z tego samego wątka.
- Zaletą jest automatyczne zwolnienie mutexa nawet gdy aplikacja się zakończy nie wywołując `ReleaseMutex`

Wait Handles

Mutex

```
static Mutex mutex = new Mutex(false, "tu.kielce.pl mutex");
static void ThreadWithMutex(object o)
    {
    //it is very slow, there is 100 times less iterations then in other
examples
    for (int ii = 0; ii < 10000; ii++)
        {
        mutex.WaitOne();
        counter++;
        mutex.ReleaseMutex();
        }
    }
```

przykład: Synchronization

Wait Handles

`Semaphore`

- Semafor jest jak licznik, który nigdy nie może być mniejszy od 0. Operacja **`WaitOne`** zmniejsza ten licznik o 1, jeżeli jest 0 to dany wątek czeka, aż inny zwiększy go za pomocą **`Release`**.
- W przypadku semafora, podnieść go może każdy inny wątek, nie tylko ten który go opuścił, tak jak to jest w przypadku *lock* czy *Mutex*.
- Semafor jest nieco szybszy od *Mutexa*.

Wait Handles

Semaphore

```
static Semaphore sem = new Semaphore(1, 1);
        static void ThreadWithSemaphore(object o)
        {
            //it is very slow, there is 100 times less iterations
            for (int ii = 0; ii < 10000; ii++)
            {
                sem.WaitOne();
                counter++;
                sem.Release();
            }
        }
```

Przykład Synchronization

Wait Handles

Wait, wait, wait...

WaitHandle.SignalAndWait – Jednoczesne wysłanie sygnału i czekanie. Można w ten sposób zrealizować np. spotkania.

```
private static EventWaitHandle wh1 =
    new
  EventWaitHandle(false,EventResetMode.AutoReset);
  private static EventWaitHandle wh2 =
    new
  EventWaitHandle(false,EventResetMode.AutoReset);
```

Jeden z wątków wywołuje:

```
WaitHandle.SignalAndWait(wh1, wh2);
```

Drugi z wątków wywołuje:

```
WaitHandle.SignalAndWait(wh2, wh1);
```

Wait Handles

`Wait, wait, wait...`

`WaitHandle.WaitAll(WaitHandle[] waitHandles)`

- Czekaj na pozwolenie od wszystkich z waitHandles

`WaitHandle.WaitAny(WaitHandle[] waitHandles)`

- Czekaj na pozwolenie od którego kolwiek z waitHandles

Bariera

Bariera jest stosowana do synchronizacji pracy kilku wątków w pewnych etapach. Przykładowo w algorytmach genetycznych gdzie czekamy, aż wszystkie wątki zakończą pracę w danej iteracji. Poniżej niezsynchronizowane praca kilku wątków.

```
const int threads = 10;
static void printString(string inputstring)
{
    ThreadStart thread = () =>
    {
        char[] inputArray = inputstring.ToArray();
        for (int i = 0; i < inputArray.Length; i++)
        {
            Console.Write(inputArray[i]);
        }
    };
    Thread[] watki = new Thread[threads];
    for (int i = 0; i < threads; ++i)
    {
        watki[i] = new Thread(thread);
        watki[i].Start();
    }
    //Here we wait until all threads running in this method are finished
    for (int i = 0; i < threads; ++i)
    {
        watki[i].Join();
    }
}
```

Example: CollectionsAndBariers/Barrier

Bariera

Próba zastosowania Monitora

```
static void printStringWaitPulse(string inputstring)
       {
           object o = new object();
           int callCounter = 0;
           ThreadStart thread = () =>
           {
               char[] inputArray = inputstring.ToArray();
               for (int i = 0; i < inputArray.Length; i++)
               {
                   lock (o)
                   {
                       Console.Write(inputArray[i]);
                       callCounter++;
                       if (callCounter < threads)
                       {   //if not all threads finished we wait
                           //and simultanously free the lock
                           Monitor.Wait(o);
                       }
                       else
                       {   //if we are the last thread we pulse others
                           Monitor.PulseAll(o);
                           callCounter = 0;
                       }
                   }
               }
           };
          ...
```

Example: CollectionsAndBariers/Barrier

Bariera

Zastosowanie *Barrier* oraz *CountdownEvent*

```
static CountdownEvent ce = new CountdownEvent(threads);
static System.Threading.Barrier barrier =
        new System.Threading.Barrier(threads, (b) =>
        { Console.WriteLine(" Barrier in a phase: {0}", b.CurrentPhaseNumber); });
static void printStringBarrier(string inputstring)
    {
    ThreadStart thread = () =>
        {
        for (int i = 0; i < inputstring.Length; i++)
            {
            Console.Write(inputstring[i]);
            barrier.SignalAndWait();
            }
        ce.Signal(); //when the task is done we report it to decrease the counter
        };
    Thread[] threadsArray = new Thread[threads];
    for (int i = 0; i < threads; ++i)
        {
        threadsArray[i] = new Thread(thread);
        threadsArray[i].Start();
        }
    //Here we wait until the counter reaches 0 (all threads calling ce.Signal ())
    ce.Wait();
    Console.WriteLine("Done. InitialCount={0}, CurrentCount={1}, IsSet={2}",
                      ce.InitialCount, ce.CurrentCount, ce.IsSet);
    }
```

Example: CollectionsAndBariers/Barrier

Kolekcje

Dodawanie do listy przez wiele wątków

```
static void Main(string[] args)
        {
            List<Thread> threads = new List<Thread>();
            List<int> numbers = new List<int>(10000);
            Random rand = new Random();
            for (int i = 0; i < 100; i++)
            {
                var thread = new Thread(() =>
                {
                    for (int l = 0; l < 100; l++)
                        numbers.Add(rand.Next());
                });
                threads.Add(thread);
                thread.Start();
            }
            foreach (var watek in threads)
            {
                watek.Join();
            }
           Console.WriteLine($"The number of items in a regular list:
{numbers.Count}");
            Console.ReadLine();
        }
```


Example: CollectionsAndBariers/UnsafeCollection

Kolekcje

Dodawanie do listy przez wiele wątków przy użyciu *lock*

```
static void Main(string[] args)
    {
    List<Thread> threads = new List<Thread>();
    List<int> numbers = new List<int>(10000000);
    Stopwatch sw = new Stopwatch();
    sw.Start();
    Random rand = new Random();
    for (int i = 0; i < 100; i++)
        {
        var thread = new Thread(() =>
            {
            for (int l = 0; l < 100000; l++)
            lock (numbers) //comment this and mesure time
                {
                numbers.Add(rand.Next());
                }
            });
        threads.Add(thread);
        thread.Start();
        }
    foreach (var th in threads)
        th.Join();
    sw.Stop();
    Console.WriteLine($"The number of items in a regular list: {numbers.Count}");
    Console.WriteLine($"Elapsed {sw.ElapsedMilliseconds} ms");
    Console.ReadLine();
    }
```


Example: CollectionsAndBariers/LockCollection

Kolekcje

Użycie ConcurrentBag

```
static void Main(string[] args)
        {
            ConcurrentBag<int> bag = new ConcurrentBag<int>();
            List<Thread> threads = new List<Thread>();
            Stopwatch sw = new Stopwatch();
            sw.Start();
            Random rand = new Random();
            for (int i = 0; i < 100; i++)
            {
                var thread = new Thread(() =>
                {
                    for (int l = 0; l < 100000; l++)
                        bag.Add(rand.Next());
                });
                threads.Add(thread);
                thread.Start();
            }
            foreach (var th in threads)
            {
                th.Join();
            }
            sw.Stop();
            Console.WriteLine($"The number of items in the concurrent bag:
{bag.Count}");
            Console.WriteLine($"Elapsed {sw.ElapsedMilliseconds} ms");
            Console.ReadLine();
        }
```

Example: CollectionsAndBariers/ConcurrentBagCollection

Kolekcje

Problem z unikalnymi wartościami

```
for (int i = 0; i < 100; i++)
            {
                var watek = new Thread(() =>
                {

                    for (int l = 0; l < 100; l++)
                    {
                        int number = rand.Next(10001);
                        lock (numbers) //we use this lock only for that
there are no errors like "collection was modified"
                            while (numbers.Any(x => x == number))
                        {
                            number = rand.Next(10001);
                        };
                        lock (numbers) //we use this lock only for that
there are no errors like "collection was modified"
                            numbers.Add(number);
                    }
                });
                threads.Add(watek);
                watek.Start();
            }
```


Example: CollectionsAndBariers/NonUniqueCollection

Kolekcje

Użycie lock, aby kolekcja była unikalna

```
for (int i = 0; i < 100; i++)
            {
                var watek = new Thread(() =>
                {

                    for (int l = 0; l < 100; l++)
                        lock (numbers)
                        {
                            int number = rand.Next(10001);
                            while (numbers.Any(x => x == number))
                            {
                                number = rand.Next(10001);
                            };
                            numbers.Add(number);
                        }
                });
                threads.Add(watek);
                watek.Start();
            }
```


Example: CollectionsAndBariers/UniqueCollection

Kolekcje

Podobny przypadek, kiedy użyjemy concurrentBag do unikalnej kolekcji.

```
ConcurrentBag<int> numbers = new ConcurrentBag<int>();
List<Thread> threads = new List<Thread>();

for (int i = 0; i < 100; i++)
    {
    var thread = new Thread(() =>
        {
        for (int l = 0; l < 100; l++)
        lock (numbers) //without this, cb has non unique numbers
            {
            int number = rand.Next(10001);
            while (numbers.Any(x => x == number))
                {
                number = rand.Next(10001);
                };
            numbers.Add(number);
            }
        });
     threads.Add(thread);
     thread.Start();
     };
```

Example: CollectionsAndBariers/NonUniqueBag

Kolekcje

Użycie ConcurrentDictionary

Klucze w słowniku muszą być unialne więc użyjemy kluczy jako wartości. Wtedy wartości mogą być null.

```
List<Thread> threads = new List<Thread>();
ConcurrentDictionary<int, object> numbers = new
ConcurrentDictionary<int, object>();
for (int i = 0; i < 100; i++)
    {
    var thread = new Thread(() =>
        {
        Random rand = new Random(Thread.CurrentThread.ManagedThreadId);
        for (int l = 0; l < 100; l++)
            {
            int number = rand.Next(10001);
            while (!numbers.TryAdd(number, null)) //we use key like a
value and value set null
                {
                number = rand.Next(10001);
                };
            }
        });
    threads.Add(thread);
    thread.Start();
    }
```

Example: CollectionsAndBariers/UniqueDictionary

Kolekcje

Zastosowanie BlockingCollection w producencie i konsumencie

```
var blockingCollection = new BlockingCollection<int>(10);
 var producer = new Thread(() =>
    {
    for (int l = 0; l < 100; l++)
        {
        Console.WriteLine($"Put  {l}");
        blockingCollection.Add(l);
        Thread.Sleep(rand.Next(500));
        }
    blockingCollection.CompleteAdding();
    });
producer.Start();

var consumer = new Thread(() =>
    {
    for (int l = 0; l < 10; l++)
        {
        var result2 = blockingCollection.Take();
        Console.WriteLine($"Take {result2} ");
        Thread.Sleep(rand.Next(5000));
        }
    });
```

Example: CollectionsAndBariers/ProducerConsumerBC

Kolekcje

Zastosowanie BlockingCollection w producencie i konsumencie

```
var blockingCollection = new BlockingCollection<int>(10);

var producer = new Thread(() =>
    {
    for (int l = 0; l < 100; l++)
        {
        Console.WriteLine($"Put  {l}");
        blockingCollection.Add(l);
        Thread.Sleep(rand.Next(500));
        }
    blockingCollection.CompleteAdding();
    });
producer.Start();

var consumer = new Thread(() =>
    {
    for (int l = 0; l < 10; l++)
        {
        var result2 = blockingCollection.Take();
        Console.WriteLine($"Take {result2} ");
        Thread.Sleep(rand.Next(5000));
        }
    });
```

Example: CollectionsAndBariers/ProducerConsumerBC

Kolekcje

Użycie zwykłej kolejki w producenciei konsumencie.

Producent wkłada kolejne liczby do zwykłej kolejki w losowych odstępach czasu od 0 do 0,5s

```
var cq = new Queue<int>();
List<Thread> consumers = new List<Thread>();
var producer = new Thread(() =>
    {
    Random rand = new Random(Thread.CurrentThread.ManagedThreadId);
    for (int l = 0; l < 100; l++)
        {
        cq.Enqueue(l);
        Console.WriteLine($"Put  {l}");
        Thread.Sleep(rand.Next(500));
        }
    });
producer.Start();
```

Example: CollectionsAndBariers/ProducerConsumerQ1

Kolekcje

Użycie zwykłej kolejki w producenciei konsumencie.

```
for (int i = 0; i < 10; i++)
    {
    var consumer = new Thread(() =>
        {
        Random rand = new Random(Thread.CurrentThread.ManagedThreadId);
        for (int l = 0; l < 10; l++)
            {
            try
                {
                int result2 = cq.Dequeue();
                Console.WriteLine($"Take {result2} ");
                }
            catch (Exception ex)
                {
                Console.WriteLine("Exception during taking: {0}", ex.Message);
                }
            Thread.Sleep(rand.Next(5000));
            }
        });
    consumers.Add(consumer);
    consumer.Start();
    }
```

Example: CollectionsAndBariers/ProducerConsumerQ1

Kolekcje

Tu mamy poważny problem. Konsument czyta dane nawet gdy kolejka jest pusta.

```
Take 0
Put  0
Exception during taking: Queue empty.
Exception during taking: Queue empty.
Exception during taking: Queue empty.
Exception during taking: Queue empty.
Exception during taking: Queue empty.
Exception during taking: Queue empty.
Exception during taking: Queue empty.
Exception during taking: Queue empty.
Exception during taking: Queue empty.
Exception during taking: Queue empty.
Put  1
Put  2
```

Example: CollectionsAndBariers/ProducerConsumerQ1

Kolekcje

Użycie Concurrent Queue w producencie i konsumencie

```
//the producer puts 100 elements into the collection at random intervals
of up to 500ms
var producer = new Thread(() =>
    {
    Random rand = new Random(Thread.CurrentThread.ManagedThreadId);
    for (int l = 0; l < 100; l++)
        {
        Console.WriteLine($"Put  {l}");
        cq.Enqueue(l);
        Thread.Sleep(rand.Next(500));
        }
    });
producer.Start();
```

Example: CollectionsAndBariers/ProducerConsumerQ2

Kolekcje

Użycie Concurrent Queue w producencie i konsumencie

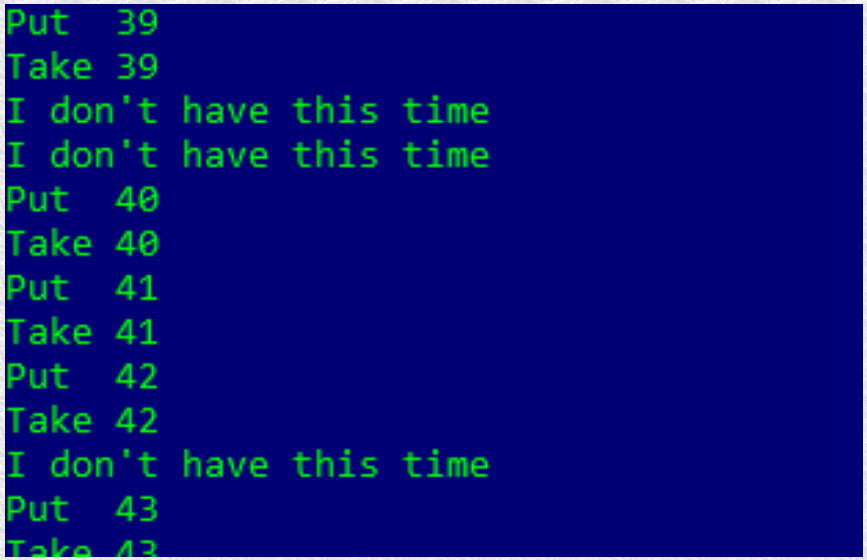
```
//Consumers take data from the collection at intervals of up to 5000 ms
for (int i = 0; i < 10; i++)
    {
    var consumer = new Thread(() =>
        {
        Random rand = new Random(Thread.CurrentThread.ManagedThreadId);
        for (int l = 0; l < 10; l++)
            {
            Thread.Sleep(rand.Next(5000));
            int result;
            bool succeeded = cq.TryDequeue(out result);
            if (!succeeded)
                {
                l--;
                Console.WriteLine("I don't have this time");
                continue;   //continue for
                }
            Console.WriteLine($"Take {result} ");
            }
        });
    consumers.Add(consumer);
    consumer.Start();
    }
```

Example: CollectionsAndBariers/ProducerConsumerQ2

Kolekcje

Mamy kolejny problem. W ConcurrentQueue brakuje blokującej funkcji pobierającej z kolejki. (W BlockingCollection jest funkcja *Take*).

Jest tylko TryDeque i jej używamy. Ale czasami mamy pustą kolejkę i trzeba wykonać ponownie TryDeque. To jest prawie jak aktywne czekanie.



Example: CollectionsAndBariers/ProducerConsumerQ2

Kolekcje

Używanie Concurrent Queue w producencie i konsumencie wraz z semaforem.

```
SemaphoreSlim elementCounter = new SemaphoreSlim(0, int.MaxValue);
var producer = new Thread(() =>
    {
    Random rand = new Random(Thread.CurrentThread.ManagedThreadId);
    for (int l = 0; l < 100; l++)
        {
        Console.WriteLine($"Put  {l}");
        cq.Enqueue(l);
        elementCounter.Release();
        Thread.Sleep(rand.Next(500));
        }
    });
producer.Start();
```

Example: CollectionsAndBariers/ProducerConsumerQ3

Kolekcje

Używanie Concurrent Queue w producencie i konsumencie wraz z semaforem.

```
for (int i = 0; i < 10; i++)
    {
    var consumer = new Thread(() =>
        {
        Random rand = new Random(Thread.CurrentThread.ManagedThreadId);
        for (int l = 0; l < 10; l++)
            {
            Thread.Sleep(rand.Next(4000));
            int result;
            elementCounter.Wait();
            bool succeeded = cq.TryDequeue(out result);
            if (!succeeded)
                {
                throw new Exception("OMG we shouldn't be here");
                }
            Console.WriteLine($"Take {result} ");
            }
        });
    consumers.Add(consumer);
    consumer.Start();
    }
```

Example: CollectionsAndBariers/ProducerConsumerQ3

Kolekcje

Używanie Concurrent Queue w producencie i konsumencie wraz z semaforem i górnym limitem.

```
SemaphoreSlim elementCounter = new SemaphoreSlim(0, limit);
SemaphoreSlim upperLimit = new SemaphoreSlim(limit, limit);

var producer = new Thread(() =>
    {
    Random rand = new Random(Thread.CurrentThread.ManagedThreadId);
    for (int l = 0; l < 100; l++)
        {
        upperLimit.Wait(); //check the queue not hit the upper limit
        Console.WriteLine($"Put  {l}");
        cq.Enqueue(l);
        elementCounter.Release(); //signal that item enqueued
        Thread.Sleep(rand.Next(500));
        }
    });
producer.Start();
```

Example: CollectionsAndBariers/ProducerConsumerQ4

Kolekcje

Używanie Concurrent Queue w producencie i konsumencie wraz z semaforem i górnym limitem.

```
Thread.Sleep(5000); //wait for simulate hiting the limit
for (int i = 0; i < 10; i++)
    {
    var consumer = new Thread(() =>
        {
        Random rand = new Random(Thread.CurrentThread.ManagedThreadId);
        for (int l = 0; l < 10; l++)
            {
            Thread.Sleep(rand.Next(4000));
            int result;
            elementCounter.Wait();
            bool succeeded = cq.TryDequeue(out result);
            if (!succeeded)
                {
                throw new Exception("OMG we shouldn't be here");
                }
            upperLimit.Release();
            Console.WriteLine($"Take {result} ");
            }
        });
    consumers.Add(consumer);
    consumer.Start();
    }
```

Example: CollectionsAndBariers/ProducerConsumerQ4

Dziękuję Państwu za uwagę.